



RAPPORT JEU DE SAMEGAME

A RENDRE POUR LE
20 AVRIL 2025

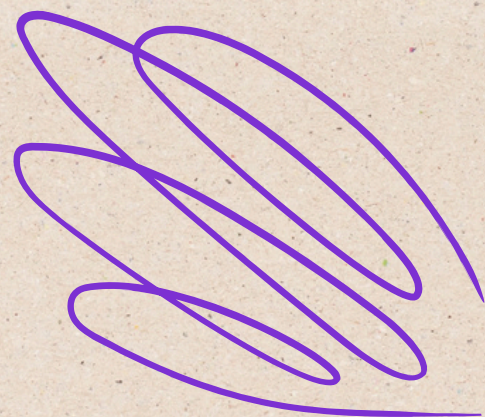


EL MCHANTEF Sarah
PEIRO TOMAS Maylee



SOMMAIRE

- 1 Introduction**
- 2 Fonctionnalités du code**
- 3 Structure et diagramme de classe**
- 4 Algorithmes et groupes**
- 5 Conclusions personnelles**

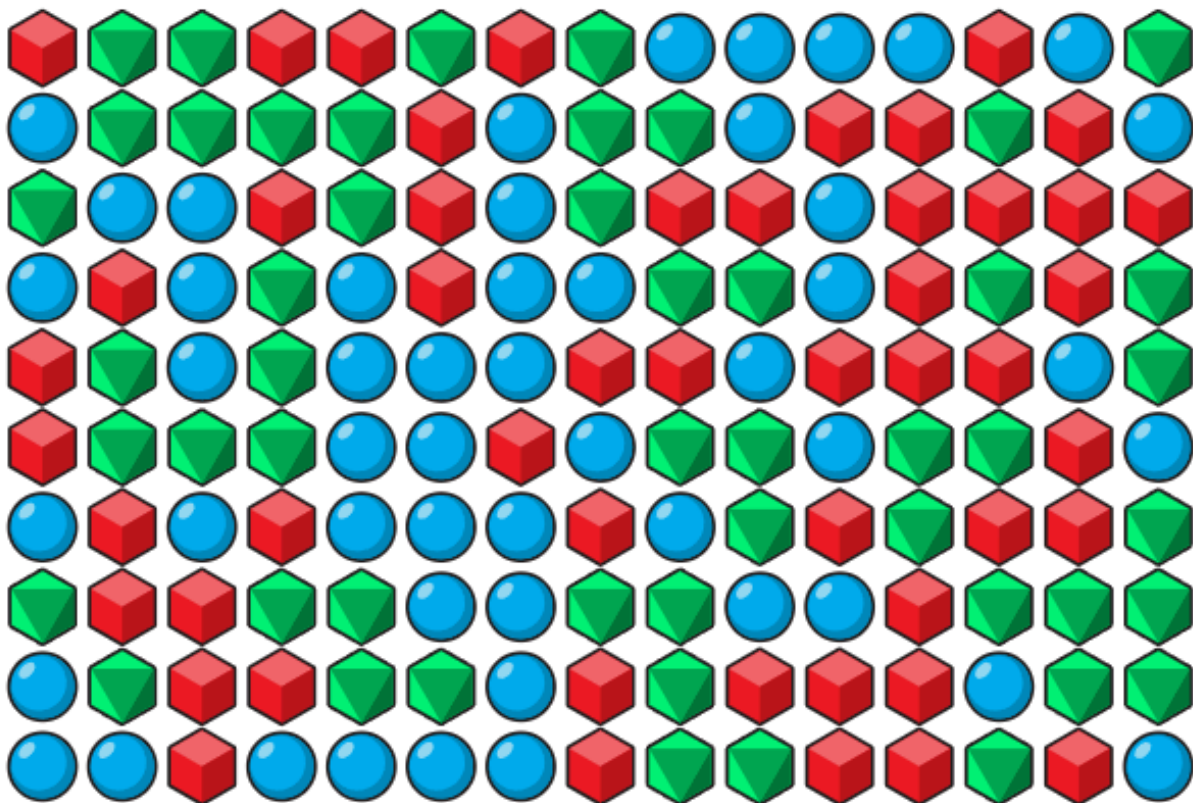


1

Introduction :

Rappel des règles du jeu :

Le principe du jeu de **SameGame** est de vider une grille constituée de 3 types de blocs de façon méthodique. En effet, les blocs de même type qui sont adjacents constituent un groupe qui peut se supprimer en cliquant dessus. Dès qu'un groupe est supprimé, les blocs des cases du dessus tombent et remplissent les cases vides de sorte à ce qu'il n'y ait pas de vide. En fonction de la taille du groupe supprimé, le score augmente différemment. Plus le groupe de blocs supprimé est grand, plus le score augmente rapidement. Evidemment, le but est d'obtenir le **score le plus élevé possible**.



Dans ce rapport nous décrivons les fonctionnalités, structures, algorithmes, etc de notre code en Java, dans le respect des consignes imposées.

2

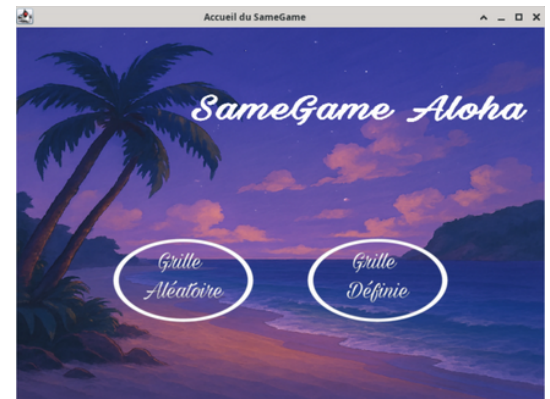
Fonctionnalités du code :

Le jeu de SameGame offre assez de règles pour y tirer de nombreuses fonctionnalités. Nous citons ici les fonctionnalités les plus intéressantes et indispensables à notre jeu de SameGame.

A) Affichage d'une page d'accueil :

La page d'accueil de notre jeu de SameGame permet au joueur de choisir le type de partie qu'il veut lancer : soit il choisit de jouer avec une grille aléatoire, soit il peut lui même choisir la disposition exacte des blocs dans la grille via un **fichier .txt**.

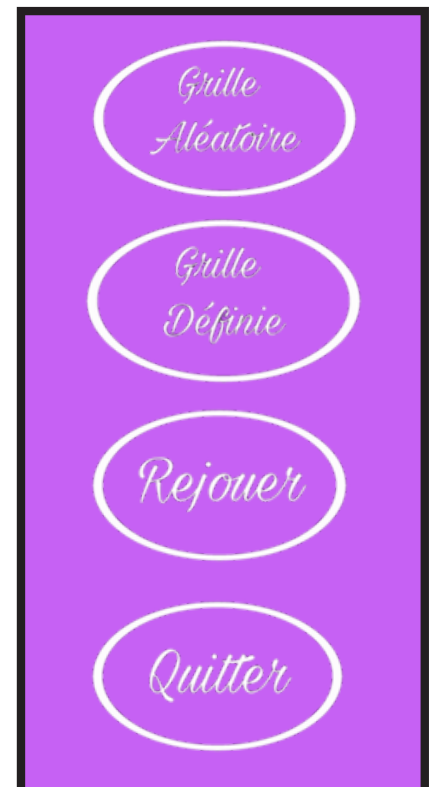
Elle est composée de la fenêtre principale **AccueilSameGame** et de deux boutons dans un JPanel secondaire. Les deux boutons utilisent un écouteur d'évènements pour déclencher l'action souhaitée.



B) Utilisation de boutons images et leurs fonctionnalités:

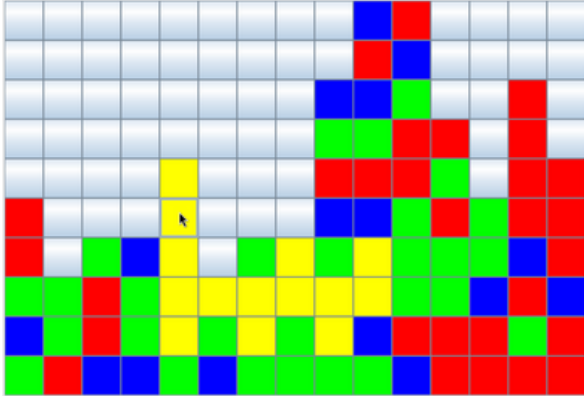
Nous avons créé 4 boutons images avec l'outil de montage Picsart. Ils se servent de l'écouteur d'évènement `MouseListener` pour rendre la page interactive :

- Le bouton **Grille Aléatoire** : Il sert à générer une grille aléatoirement. Lorsqu'il est cliqué, l'écouteur appelle la méthode **LancerJeuAleatoire**. Ainsi, une instance de **Grille** est créée et la grille est remplie de blocs de façon aléatoire grâce à la classe **GenererGrilleAleatoire**.
- Le bouton **Grille Définie** : l'écouteur d'évènement ouvre le `JFileChooser` et permet à l'utilisateur de choisir un fichier qui définira la disposition des blocs de sa grille. La méthode **LireGrilleDepuisFichier** est appelée et interprète le fichier texte constitué de R, V et B (respectivement rouge, vert et bleu) pour remplir la grille des images correspondantes grâce à la classe **SetBloc**.
- Le bouton **Rejouer** : Il est placé sur la fenêtre de fin de jeu et sert à revenir à l'écran d'accueil. Le score est remis à zéro et le jeu est réinitialisé. La méthode **AccueilSameGame** est appelée et initialisée par un **new AccueilSameGame()**;
- Le bouton **Quitter** : Il sert à fermer le jeu de SameGame. Il appelle simplement la méthode **System.exit(0)**;



C) Faire apparaître les images dans la grille :

Notre code nous permet de remplir les cases de notre grille de blocs de trois types différents : rouge, vert ou bleu. Initialement ces blocs étaient représentés en coloriant les cases grâce à la méthode **SetBackground**.

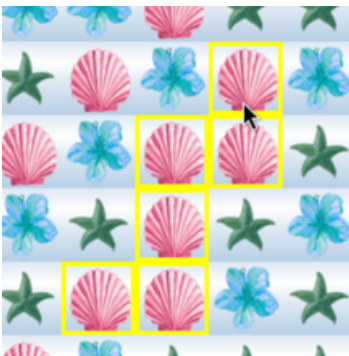


Or, cette façon de remplir la grille n'était que temporaire avant de pouvoir placer des images dans notre grille grâce à la méthode **SetIcon** qui est appelée dans la classe **RaffraichirGrille**.



Notre grille est un **tableau d'entiers** : il est rempli par trois valeurs possibles (1,2 et 3 pour rouge, vert et bleu). Selon l'entier contenu dans le tableau, une image y est placée. Pour rouge, un coquillage rose, pour bleu, une fleur bleue et pour vert, une étoile de mer verte. Pour une valeur différente de 1,2 ou 3, la case n'affiche aucune image. Ces correspondances sont gérées dans la classe **GetCouleur** (la classe gère aussi la taille des images à 50 pixels sur 50 pour qu'elles s'affichent bien). Ainsi, lorsque la grille est mise à jour, on appelle la méthode **SetIcon** pour afficher les images dans la grille.

D) Surligner les groupes :



Comme demandé dans la consigne, il est possible de mettre en valeur les groupes de blocs adjacents de même couleur. Pour cela, il faut d'abord identifier les groupes.

L'écouteur d'événements **MouseMotionListener** détecte le survol de la souris : chaque mouvement de la souris déclenche **MouseMoved()** et la méthode **survolerBloc(x,y)** de la classe **SurvolerBloc** est appelée.

Si la case contient un bloc, elle va regarder si les blocs adjacents contiennent la même valeur (1,2 ou 3). Puis elle fait de même pour la case voisine de même valeur.

La méthode **SurvolerGroupe.survolerGroupe(int ligne, int colonne, int couleur, boolean[][] visite)** effectue une recherche récursive : à partir du bloc que la souris survole, elle regarde la case du haut, du bas, de gauche et de droite. Si la case analysée est de même couleur alors elle part de cette case là et effectue la même recherche et ainsi de suite. Il s'agit là d'un **parcours en profondeur**. Le tableau booléen '**visite**' est utilisé pour éviter de repasser sur une case déjà visitée et évite donc les boucles infinies.

Ainsi, la méthode **Raffraichirgrille.raffraichir()** est appelée. Elle parcourt toute la grille et lorsqu'elle détecte un groupe, elle applique une bordure jaune avec **BorderFactory.createLineBorder(Color.YELLOW, 3)**.

E) Afficher un score actuel et final :

Au cours de la partie, le score (initialisé à 0 au début dans la classe **BaseScorePanel**) augmente en fonction des suppressions des groupes de la grille par le joueur. La variable **score** dans la classe **BaseScorePanel** est un entier qui garde en mémoire le score du joueur. Il augmente plus ou moins rapidement selon la taille des groupes que le joueur supprime. Le score est mis à jour grâce à la méthode **ScorePanel.addPoints** qui additionne au fur et à mesure les points accumulés pour former le score.

L'affichage du score est donc mis à jour avec la méthode **updateScore()**. C'est grâce à la classe **ScorePanel** que le score est affiché en haut de la grille. Elle contient un champs **score** pour stocker la valeur et un composant Swing **JLabel** pour l'afficher dans la zone NORTH de la fenêtre. Enfin, le score est contenu dans un panneau **JPanel**.

Score : 144

page de jeu

Score final : 1156

page de fin

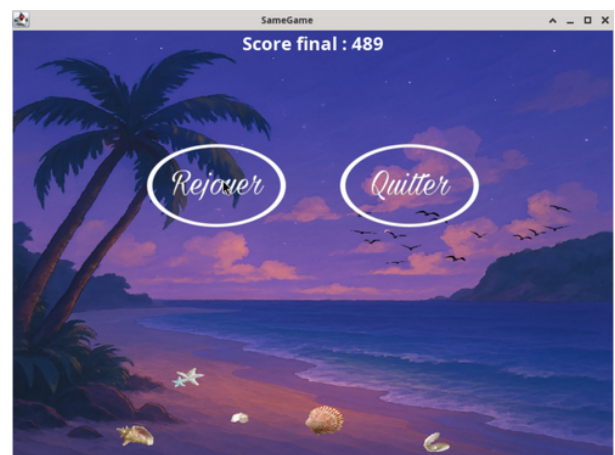
Sur la page de fin, le score final est affiché avec la classe **SuperpositionFinDePartie** à l'aide d'un label. La variable **scoreFinal** récupère le score en fin de partie et permet de l'afficher.

F) Affichage d'une page de fin :

Dans la classe **DetecterEtSupprimerGroupe**, la méthode **verifierSiFinDePartie()** parcourt le tableau pour voir si il reste des groupes supprimables ou non. Si il ne reste aucun groupe, la méthode retourne **true**.

Dès que la méthode retourne **true**, la partie est terminée et la page de fin s'affiche.

Cela se fait avec un **new SuperPositionFinDePartie** dans la classe **DetecterEtSupprimerGroupe**. Il s'agit d'une superposition (**glassPane**) : le fond image recouvre la grille et le panneau de score s'affiche (version fin de partie).



3

Structure et diagramme de classe

A) Structure et héritage :

Notre code est organisé de la façon la plus claire possible : une classe par fichier. Cela fait donc un total de **38 fichiers .java** avec un **makefile** :

1. TailleGroupe.java	21. FenetreFinDePartie.java
2. SurvolerGroupe.java	22. FenetreDeBase.java
3. SurvolerBloc.java	23. FaireTomberLesBlocs.java
4. SupprimerGroupe.java	24. EstGroupeValide.java
5. SupprimerBloc.java	25. DetecterEtSupprimerGroupe.java
6. SuperpositionFinDePartie.java	26. DeplacerBloc.java
7. SetBloc.java	27. DecalerColonnes.java
8. ScorePanel.java	28. ComposantFond.java
9. SameGame.java	29. ChargerGrilleDepuisFichier.java
10. RaffraichirGrille.java	30. AccueilSameGame.java
11. PanneauBoutonsFinDePartie.java	31. BoutonSamegame.java
12. Main.java	32. BaseScorePanel.java
13. LireGrilleDepuisFichier.java	33. AfficherFinDePartie.java
14. LancerJeuAleatoire.java	34. BoutonAleatoire.java
15. InitGrille.java	35. BoutonChoix.java
16. InitGame.java	36. BoutonQuitterFin.java
17. Grille.java	37. BoutonRejouerFin.java
18. GetCouleur.java	38. GrilleBoutonController.java
19. GenererGrilleAleatoire.java	
20. FinDePartie.java	+ un makefile

Chacune de nos **38 classes** est responsable d'un **aspect précis** :

- Modèle de la grille : Grille, SetBloc, DeplacerBloc, SupprimerBloc.
- Logique de groupes : TailleGroupe, SurvolerGroupe, SurvolerBloc, EstGroupeValide, ...
- Réorganisation après suppression : FaireTomberLesBlocs, DecalerColonnes.
- Affichage et score : FenetreDeBase, ComposantFond, BaseScorePanel, ScorePanel.
- Écran d'accueil et contrôleurs : AccueilSameGame, Main, InitGame, InitGrille, ...

Les 4 boutons “**Grille Aléatoire**”, “**Grille Définie**”, “**Rejouer**” et “**Quitter**” sont liés à des contrôleurs qui permettent de déclencher différentes actions. Ils sont implémentés via **MouseListener**. Pour les boutons, seulement **MouseClicked()** est utilisé et chaque contrôleur s’occupe de la méthode qui va être appelée ensuite selon le bouton :

BoutonAleatoire → appelle `LancerJeuAleatoire.lancerJeuAleatoire(...)`.

BoutonChoix → appelle `ChargerGrilleDepuisFichier.chargerGrilleDepuisFichier(...)`.

BoutonRejouerFin → ferme la fenêtre courante et fait `new AccueilSameGame()`.

BoutonQuitterFin → exécute `System.exit(0)`.

De plus, chaque classe est accompagnée d’une documentation **Javadoc** inspirée de l’exemple de la consigne. Cela permet au correcteur, mais à nous aussi, de comprendre l’utilité de chacune de nos classes et de ses paramètres.

Enfin, nous avons deux cas de figure pour lesquels nous avons trouvé intéressant de faire de l’**héritage** entre nos classes. En effet, l’héritage permet de rassembler du code commun dans une classe spéciale qui va permettre de ne pas se répéter dans les classes qui en héritent :

-Les classes **FenetreFinDePartie**, **InitGame** et **AccueilSameGame** héritent de la classe **FenetreDeBase**, cela permet d’avoir la même disposition de la fenêtre de jeu. Les trois fenêtres ont donc une apparence cohérente et similaire grâce à **FenetreDeBase**, ce qui rend le jeu uniforme.

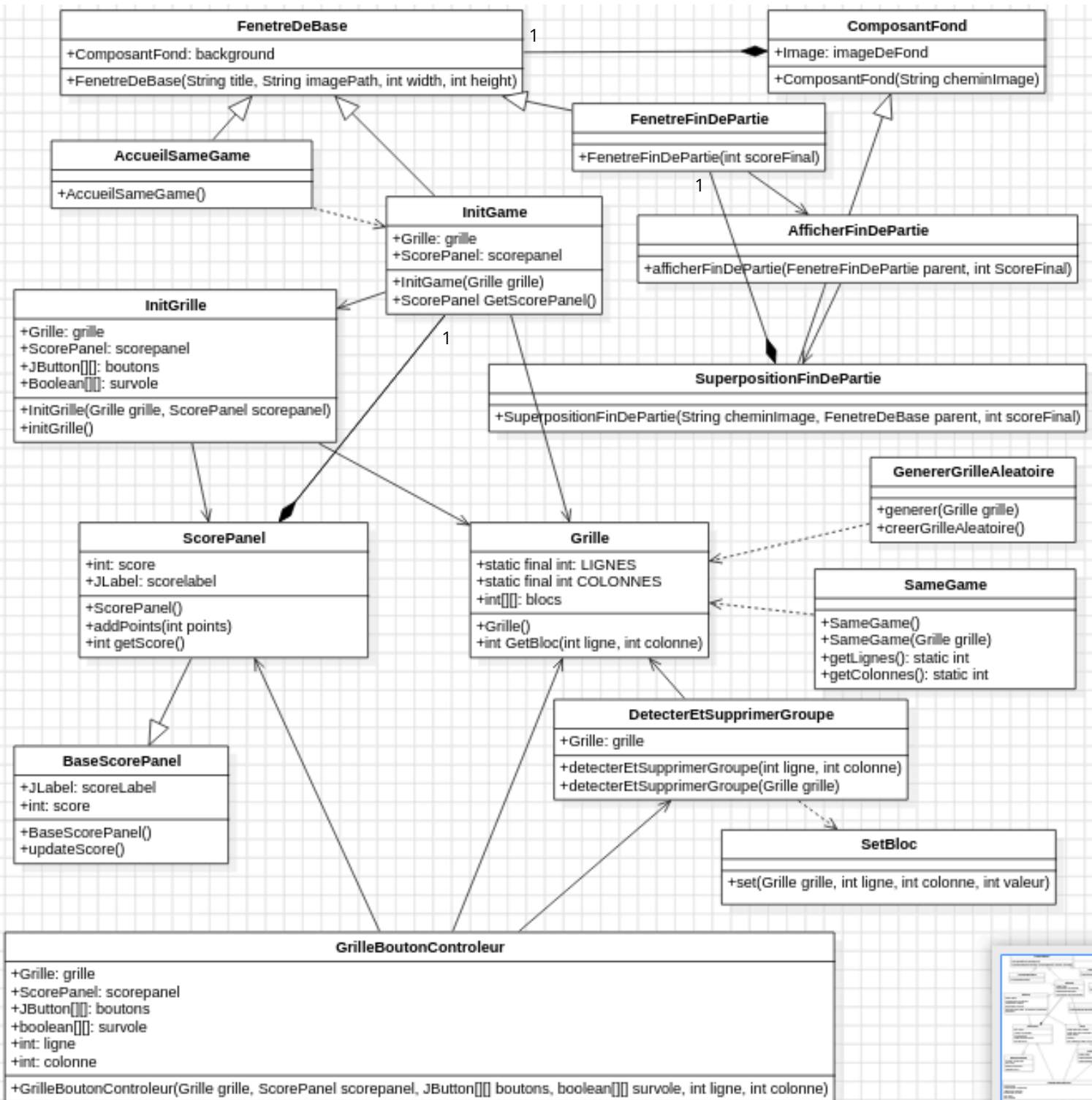
-La classe **ScorePanel** qui hérite de **BaseScorePanel**, ceci permet à **ScorePanel** d’hériter du style d’affichage du panneau de score de **BaseScorePanel**. Initialement nous avons une classe **ScorePanelFin** qui affichait le score final sur la page de fin en héritant aussi de **BaseScorePanel**, or, nous nous sommes rendues compte que le panneau pour le score de fin ou de début pouvait être contenu dans une même classe.

Nous avons tout de même décidé de garder cette notion d’héritage avec **BaseScorePanel** car il pourrait être intéressant dans le futur. Par exemple, si nous décidons d’afficher un message du style “Reccord à battre : 1450 !” sur la page d’accueil pour motiver l’utilisateur à jouer, nous aurions seulement à ajouter une classe que nous appellerions par exemple **MeilleurScore** qui hériterait de **BaseScorePanel** pour garder le style d’affichage du jeu.

L’héritage nous permettrait aussi d’effectuer des modifications plus rapidement en modifiant seulement la classe qui fait hériter les autres sans avoir à changer plusieurs fichiers. Par exemple, si nous souhaitons modifier la **police d’écriture** du score, il nous suffit de modifier **BaseScorePanel** et non pas toutes les classes affichant un score sur chaque fenêtre.

B) Diagramme de classes :

Voici le diagramme de classe de notre code principal. Etant donné que nous avons énormément de classes, créer un diagramme avec absolument toutes les classes n'aurait pas été intéressant (aussi et surtout illisible). Nous avons donc décidé de le simplifier comme ceci de sorte à pouvoir montrer les relations d'association, de généralisation (héritage) et de composition.



4

Algorithmes et groupes

Dans notre jeu, l'algorithme que nous avons conçu détecte automatiquement le nombre de blocs reliés de même couleur lorsqu'on survole ou clique sur une case. Il vérifie que ce groupe contient au moins deux blocs adjacents et le supprime lorsqu'on clique dessus.

Initialisation et préparation :

Lorsque l'utilisateur survole ou clique sur une case, on commence par créer un tableau de booléens de la même taille que la grille, appelé tableau « visite ». Chaque case y est initialement marquée comme « non visitée ». Ce tableau nous permet de mémoriser les cases déjà analysées, évitant ainsi de repasser plusieurs fois sur la même cellule et de rester efficacement concentrés uniquement sur les cellules concernées par le groupe actuel.

Le cœur de l'algorithme (parcours en profondeur) :

L'algorithme principal fonctionne sur un principe appelé « parcours en profondeur » (ou flood-fill). À partir de la case active (survolée ou cliquée), l'algorithme examine chacune de ses quatre voisines directes (haut, bas, gauche, droite). Pour chaque case voisine, on vérifie précisément ces trois conditions :

- Elle est bien située dans les limites de la grille.
- Elle n'a pas déjà été marquée comme visitée.
- Sa couleur (1, 2 ou 3) correspond exactement à celle de la première case choisie.

Si toutes ces conditions sont vérifiées, alors cette nouvelle case est marquée comme visitée et ajoutée au groupe, et on poursuit l'exploration à partir de cette nouvelle cellule. C'est un peu comme dessiner un réseau invisible entre toutes les cellules voisines qui partagent la même couleur.

La classe TailleGroupe se charge concrètement d'effectuer cette recherche récursive. Son rôle est de compter chaque case connectée à la première et de s'assurer qu'on ne visite jamais deux fois la même cellule grâce au tableau « visite ».



Validation du groupe détecté :

À la fin de l'exploration, on compte simplement le nombre total de cases visitées pour déterminer la taille exacte du groupe détecté. Si ce nombre est inférieur à deux, alors on ne considère pas cela comme un groupe valide (un seul bloc isolé n'étant pas un groupe). Dans ce cas, aucun effet visuel particulier n'est appliqué lors du survol, et aucun bloc n'est supprimé au clic.

Cette vérification est effectuée par la classe EstGroupeValide.

Suppression et réorganisation :

Si le groupe est valide (taille supérieure ou égale à deux), alors au clic on parcourt à nouveau le même ensemble de blocs précédemment détecté pour supprimer chacun des blocs du groupe en les remplaçant par une case vide (valeur zéro).

Après cette suppression, deux étapes de réorganisation sont appliquées immédiatement :

- Effondrement vertical : les blocs situés au-dessus des cases supprimées « tombent » vers le bas afin de remplir les espaces vides créés.
- Compression horizontale : les colonnes restantes sont ensuite déplacées vers la gauche pour éliminer complètement les colonnes devenues totalement vides.

Ces deux étapes assurent que la grille conserve toujours un aspect compact et sans trous après chaque suppression, comme demandé dans l'énoncé.



5

Conclusions personnelles

Maylee

Encore une fois, la SAé me permet de progresser beaucoup plus vite que les TP car ils me demandent beaucoup de concentration en cours, ce qui est plus difficile que de travailler chez moi. Durant la réalisation du projet de SameGame, je me suis très rapidement familiarisée avec l'API Java et j'ai intégré de meilleures façons de raisonner comme avec l'algorithme en profondeur. De plus, contrairement au projet Blocus, nous nous y sommes mises dès le départ, ce qui nous a permis de soigner au maximum notre code. Nos problèmes principaux ont été la division de notre code car comme demandé, il fallait diviser un maximum le code source. Cela a pris un peu de temps, Ozvann nous a aidé de bon coeur d'ailleurs. En fait, notre code était relativement divisé puis nous avons entendu le professeur dire que plus les fichiers étaient courts et simples, mieux c'était. Alors on a fait de notre mieux pour avoir des classes concises. Finalement, le projet Java nous aura confronté à des difficultés inattendues de débogage.

Sarah

En réalisant ce projet, on a vraiment eu l'occasion de mettre en pratique tout ce qu'on a appris pendant le semestre, que ce soit en TD, en TP. C'était la première fois qu'on devait créer un vrai jeu en java, de A à Z, et ça change tout par rapport à un simple exercice avec un énoncé. On avait la liberté de faire nos propres choix, de construire l'interface comme on le voulait, de choisir les images, et d'organiser notre code à notre manière.

On a retrouvé beaucoup de notions vues en cours : les listeners pour les clics, la gestion des interfaces avec les JPanel, l'héritage entre classes, et même les parcours récursifs qu'on avait vus en maths. Mais là, on les a appliqués dans un vrai jeu et ça nous a permis de mieux comprendre comment tout ça fonctionne ensemble.

On a aussi rencontré des difficultés. Le découpage en plusieurs fichiers au début nous a posé pas mal de questions : comment bien organiser les classes ? Quelles données garder accessibles ? Et puis il y a eu des bugs, parfois très simples mais parfois très étranges. Le surlignage des groupes, par exemple, nous a demandé pas mal d'essais pour qu'il soit fluide et réactif.

Malgré tout, c'était super motivant, parce qu'on voyait notre jeu évoluer au fur et à mesure. À la fin, on avait quelque chose de visuellement agréable, qui fonctionne bien. Ce projet nous a permis de progresser techniquement, mais aussi de gagner en autonomie, et en organisation

